

3. Solve 8-Puzzle using Greedy Best First Search and analyse

```
import heapq

def print_board(state):
    for i in range(0, 9, 3):
        print(state[i:i+3])
    print()

def get_neighbors(state):

    neighbors = []

    zero = state.index(0)

    moves = {
        0: [1, 3],
        1: [0, 2, 4],
        2: [1, 5],
        3: [0, 4, 6],
        4: [1, 3, 5, 7],
        5: [2, 4, 8],
        6: [3, 7],
        7: [4, 6, 8],
        8: [5, 7]
```

```
}
```

```
for pos in moves[zero]:
```

```
    new_state = state.copy()
```

```
    new_state[zero], new_state[pos] = \
        new_state[pos], new_state[zero]
```

```
    neighbors.append(new_state)
```

```
return neighbors
```

```
def heuristic(state, goal):
```

```
    distance = 0
```

```
    for i in range(9):
```

```
        if state[i] == 0:
```

```
            continue
```

```
    current_row = i // 3
```

```
    current_col = i % 3
```

```
goal_index = goal.index(state[i])
```

```
goal_row = goal_index // 3
```

```
goal_col = goal_index % 3
```

```
distance += abs(current_row - goal_row)
```

```
distance += abs(current_col - goal_col)
```

```
return distance
```

```
def greedy_best_first(start, goal):
```

```
    priority_queue = []
```

```
    h = heuristic(start, goal)
```

```
    heapq.heappush(
```

```
        priority_queue,
```

```
        (h, start, [start])
```

```
    )
```

```
    visited = set()
```

```
while priority_queue:

    h, state, path = heapq.heappop(priority_queue)

    if tuple(state) in visited:
        continue

    visited.add(tuple(state))

    if state == goal:
        return path

    for next_state in get_neighbors(state):

        if tuple(next_state) not in visited:

            h = heuristic(next_state, goal)

            heapq.heappush(
                priority_queue,
                (h, next_state, path + [next_state])
            )

return None
```

```
start = [1, 2, 3,  
         4, 0, 6,  
         7, 5, 8]
```

```
goal = [1, 2, 3,  
        4, 5, 6,  
        7, 8, 0]
```

```
solution = greedy_best_first(start, goal)
```

```
if solution:
```

```
    print("Solution using Greedy Best First Search")
```

```
    print("Number of moves:", len(solution) - 1)
```

```
    for state in solution:
```

```
        print_board(state)
```

```
else:
```

```
    print("No solution found")
```

The heuristic used here is **Manhattan Distance**:

$$h(n) = \sum |x_{current} - x_{goal}| + |y_{current} - y_{goal}|$$

The algorithm chooses the state having the **smallest h(n)**.

Greedy Best-First Search uses:

$$f(n) = h(n)$$

It asks:

"Which state looks closest to the goal?"

It does **not** consider how many moves have already been made.